

Informe de Análisis Algorítmico y Empírico: Sistema de Gestión de Paquetes Logísticos

Asignatura: Análisis de Algoritmos
Institución: Universidad EAFIT
Plataforma de Entrega: EAFIT Interactiva
Repositorio de GitHub: <https://github.com/Scardenaa3/Analisisproyecto1>

Integrantes y Distribución de Responsabilidades

Integrante	Rol y Responsabilidades
Santiago Cárdenas	Diseñador Principal y Desarrollador — Implementación del algoritmo Merge Sort Iterativo (Bottom-Up), arquitectura del módulo de medición de tiempos () e integración general.
Viliam Sofía Mesa	Analista de Datos y Desarrolladora — Implementación del módulo de Búsqueda Lineal, manejo de escenarios de prueba (casos borde) y validación de la generación aleatoria de paquetes.
Mariana Cardona	Desarrolladora de Estructuras — Implementación del algoritmo Selection Sort intercambiando enlaces nativos sobre listas enlazadas simples y elaboración de pruebas unitarias.

1. Introducción y Descripción del Problema

El proyecto aborda la simulación y gestión eficiente de paquetes en un centro de distribución logístico mediante listas enlazadas simples en lenguaje C. El objetivo fundamental es evaluar el rendimiento de dos paradigmas de diseño algorítmico (Fuerza Bruta vs. Dividir y Conquistar) al procesar volúmenes masivos de datos (N = 50.000 paquetes), así como analizar las limitaciones estructurales en las operaciones de búsqueda sobre estructuras dinámicas de acceso secuencial.

2. Modelado de Datos y Algoritmos Implementados

2.1. Estructura de Datos Base

Cada paquete en el sistema se modela con la estructura Paquete (que contiene identificador único ID, peso en kilogramos y prioridad de envío del 1 al 5) y se

almacena dinámicamente mediante punteros en un Nodo de lista enlazada.

2.2. Algoritmos de Ordenamiento

Selection Sort por Enlaces Nativos (Fuerza Bruta): Busca iterativamente el nodo con el menor ID dentro de la sublista no ordenada y ajusta los punteros siguiente de los nodos involucrados para reposicionarlo. No intercambia los valores contenidos en la estructura, sino la memoria física mediante la reconfiguración estricta de sus enlaces. Complejidad: $O(N^2)$.

Merge Sort Iterativo Bottom-Up (Dividir y Conquistar): Evita la recursión para controlar el uso del call stack. Inicia fusionando sublistas de tamaño 1, 2, 4, 8, etc., cortando los enlaces y mezclándolas de forma ordenada en una lista auxiliar mediante una función iterativa de combinación. Opera nativamente sobre la lista simple sin requerir arreglos auxiliares. Complejidad: $O(N \log N)$.

2.3. Algoritmo de Búsqueda y Análisis de Limitación Estructural

Búsqueda Lineal: Recorre secuencialmente la lista nodo por nodo desde la cabeza (head) hasta hallar el ID o alcanzar NULL, con complejidad $O(N)$.

Análisis de Limitación de Búsqueda Binaria en Listas Enlazadas: Aunque la lista se encuentre perfectamente ordenada mediante Merge Sort, no es viable implementar una Búsqueda Binaria de orden $O(\log N)$ de manera nativa y eficiente. Esto se debe a que las listas simples carecen de acceso aleatorio por índice $O(1)$. Acceder al elemento medio requiere recorrer $N/2$ nodos, lo que transforma la ecuación de recurrencia en $T(N) = T(N/2) + O(N)$, degradando la complejidad de la búsqueda a $O(N)$.

3. Experimentos Empíricos y Tiempos de Ejecución

La simulación se ejecutó sobre un entorno Linux / WSL con una muestra masiva de 50.000 paquetes generados aleatoriamente.

Operación	Algoritmo / Método	Paradigma / Complejidad	Tiempo Medido (ms)
Ordenamiento	Merge Sort Iterativo	Dividir y Conquistar — $O(N \log N)$	6.50 ms
Ordenamiento	Selection Sort por Enlaces	Fuerza Bruta — $O(N^2)$	5198.58 ms (~5.2 s)
Búsqueda Individual	Búsqueda Lineal (Elemento Existente)	Recorrido Secuencial — $O(N)$	< 0.5 ms

Búsqueda Individual	Búsqueda Lineal (ID Inexistente -9999)	Recorrido Secuencial — $O(N)$	< 0.5 ms
Búsqueda Masiva	1000 Rondas (50% Éxito / 50% Fallo)	Recorrido Secuencial — $O(N)$	356.78 ms

4. Discusión de Resultados y Hallazgos

1. Eficiencia en Ordenamiento: La medición empírica valida plenamente la teoría algorítmica: Merge Sort (6.50 ms) fue aproximadamente 800 veces más rápido que Selection Sort (5198.58 ms) sobre un conjunto de 50.000 elementos. La diferencia entre un crecimiento cuadrático N^2 ($\sim 2.5 \times 10^9$ comparaciones) y un crecimiento logarítmico $N \log_2 N$ ($\sim 7.8 \times 10^5$ comparaciones) es abrumadora en entornos de volumen masivo.

2. Costo de Modificación de Enlaces vs. Arreglos: La implementación de Selection Sort intercambiando punteros directamente en la lista evita copias en memoria de las estructuras completas, pero introduce operaciones adicionales de punteros que acentúan la degradación temporal a medida que N crece.

3. Inviabilidad de Búsqueda Binaria Secuencial: Las mediciones en el módulo de búsqueda masiva (356.78 ms para 1000 consultas) confirman que el costo de búsqueda acumulado en listas secuenciales depende del recorrido de enlaces. Para lograr consultas $O(\log N)$ en sistemas de paquetes dinámicos, se requeriría migrar hacia estructuras jerárquicas avanzadas (como Árboles Binarios de Búsqueda AVL / Red-Black) o Tablas Hash $O(1)$.

5. Conclusiones

Rendimiento Algorítmico: El paradigma Dividir y Conquistar (Merge Sort) es el estándar recomendado para el procesamiento masivo en sistemas de logística industrial, demostrando una escalabilidad superior frente a alternativas de Fuerza Bruta.

Elección de Estructuras de Datos: Las listas enlazadas simples son ideales para operaciones dinámicas de inserción y eliminación sin reorganización de memoria fija $O(1)$ al inicio, pero presentan severas restricciones en rendimiento para operaciones de búsqueda masiva debido a su naturaleza de acceso secuencial.

Gestión de Memoria: El desarrollo en C exige un control riguroso de memoria dinámica (malloc / free), garantizando la liberación total de los nodos clonados y procesados en cada simulación.